



西安交通大学
XI'AN JIAOTONG UNIVERSITY

形式语言与编译 实验报告

简易编译器 QCompiler 的实现及验证

课程名称：形式语言与编译

实验名称：简易编译器 QCompiler 的实现及验证

姓 名：尤颢辰

班 级：计算机 2303

学 号：2236115104

指导教师：赵银亮

2026 年 6 月 7 日

目录

1 实验简介	3
2 实验目的	3
3 实验环境	3
4 步骤一：QL 词法分析器实现	3
4.1 任务概述	3
4.2 QL 词法单元设计	4
4.2.1 词法规则	4
4.2.2 Token 类型表	4
4.2.3 Token 数据结构	4
4.3 词法分析的 DFA 设计	5
4.3.1 总体思路	5
4.3.2 顶层状态转移	5
4.3.3 各子自动机状态转移	5
4.4 编程实现与测试	6
5 步骤二 (前半)：SLR (1) 分析框架	6
5.1 任务概述	6
5.2 QL 文法的 SLR(1) 改造	7
5.2.1 改造原则	7
5.2.2 表达式与条件的优先级分层	7
5.2.3 其他修剪	7
5.2.4 完整文法	8
5.3 SLR(1) 分析表的构造与验证	8
5.3.1 构造流程	8
5.3.2 生成 SLR(1) 分析表	9
5.3.3 冲突的发现与消解	9
5.4 表驱动分析器的实现	10
5.5 QAST 的设计与构建	10
5.6 QAST 输出测试	10
6 步骤二 (后半)：中间代码生成 (符号表 + QTAC)	11
6.1 任务概述	11
6.2 符号表设计	11
6.3 QTAC 指令与表示	12
6.4 遍历生成 QTAC 代码和符号表	12
6.4.1 遍历规则	12

6.4.2	表达式与赋值语句生成	12
6.4.3	控制流语句生成	13
6.4.4	函数、调用与声明	13
6.4.5	符号表与 QTAC 代码输出测试	13
7	步骤三：QTAC→ARM64 汇编生成	14
7.1	任务概述	14
7.2	寄存器分配	14
7.3	函数与调用约定	15
7.4	内存访问与全局数组	15
7.5	输出测试	15
8	步骤四：ARM 汇编代码优化	16
8.1	任务概述	16
8.2	基本块划分	16
8.3	中端优化：公共子表达式消除、复制传播与死代码删除	16
8.4	后端优化：冗余跳转消除	17
8.5	结果验证	17
9	步骤五：整体测试与鲲鹏环境验证	18
10	个人总结	19
10.1	AI 使用总结	19
10.2	收获与展望	20

1 实验简介

本次实验要完成简易编译器 QCompiler 实现及验证，把一门自定义的源语言 QL 经过完整的编译流程，最终翻译为 ARM64 汇编，并在合适的平台上汇编、链接、运行，验证生成代码的正确性。

大作业 1 完成了编译器后端的一段（QTAC 中间代码 → ARM64 汇编），本次实验则要把整条编译流水线从头到尾完成一遍：从源程序的字符流出发，依次经过词法分析、语法分析、中间代码生成、目标代码生成与优化，直到在真实 ARM 硬件上能够运行。

2 实验目的

本次实验旨在通过完整实现一个简易编译器，达到以下目的：

1. **掌握编译程序的完整实现要点：**贯通词法分析、语法分析、抽象语法树、符号表、中间表示、运行时环境、目标代码生成与优化各个阶段，理解它们如何协同构成一条完整的编译流水线。
2. **衔接编译理论与工程实现：**把课堂上的形式化理论（正则与自动机、上下文无关文法、语法制导翻译等）落地为可运行的代码，在实践中加深对理论的理解与抽象能力。
3. **以测试程序验证各阶段正确性：**使用 qsort 快速排序程序作为统一测试用例，逐阶段检验词法、语法、中间代码与目标代码的输出是否正确。
4. **在真实硬件上验证目标代码：**在华为鲲鹏 ARM64 环境完成汇编、链接与运行，确认生成的程序功能正确，从而验证整个编译器的有效性。

3 实验环境

本实验分为编译器开发环境与目标程序运行环境两部分。

1. QCompiler 开发环境

QCompiler 使用 Python 实现，在本地 PC（Windows）上完成编写与调试。Python 解释器版本为 3.14。

2. 目标程序运行环境

生成的 ARM64 汇编在华为鲲鹏服务器上完成汇编、链接与运行验证。

4 步骤一：QL 词法分析器实现

4.1 任务概述

词法分析是编译流程的第一个阶段，其任务是吧 QL 源程序的字符流切分为一个个具有独立含义的词法单元，并标注每个单元类别。它过滤掉空白与注释等无意义的字符，把诸如 `int arr[10];` 这样的原始字符序列识别成（关键字, int）、（标识符, arr）、（左中括号, [）等带类型的记号，作为后续语法分析的输入。

本阶段的输入是一份 QL 源程序，输出是（类型, 值）格式的 Token 序列。其中“类型”指明该记号属于哪一类词法单元，“值”保留其原始词素，供后续阶段使用。

我们使用快速排序程序 `qsort.ql` 作为测试用例，词法分析器读入该源文件，输出对应的 Token 序列到 `tokens.txt`，作为本阶段成果与下一阶段语法分析的输入。

4.2 QL 词法单元设计

4.2.1 词法规则

根据 QL 文法，源程序中出现的词法单元可归纳为五类：

1. **关键字**：语言保留字，包括 `int`、`void`、`if`、`else`、`while`、`return`；
2. **标识符**：变量名与函数名，文法规定为 $d = [a-zA-Z]^+$ ，即一个或多个字母；
3. **整数常量**：文法规定为 $i = [0-9]^+$ ，即一个或多个数字；
4. **运算符**：包括算数与赋值运算符、关系运算符与逻辑运算符；
5. **界符**：包括 `( )`、`[ ]`、`{ }`、`, , ;`；用于分隔与界定语法结构。

关键字在形式上与标识符相同，因此我们实现时统一按标识符规则识别，再查关键字表区分二者。

4.2.2 Token 类型表

为覆盖 QL 文法的全部终结符，设计如下 Token 类型。每个类型旁标注了它对应的文法终结符或词素：

表 1: QL 语言的 Token 类型设计

大类	Token 类型	对应文法终结符 / 词素
关键字	INT VOID IF ELSE WHILE RETURN	<code>int void if else while return</code>
标识符	ID	$d = [a-zA-Z]^+$
整常量	NUM	$i = [0-9]^+$
算术/赋值	PLUS MINUS MUL DIV ASSIGN PLUS_ASSIGN	<code>+ - * / = +=</code>
关系	LT LE GT GE EQ NE	<code>< <= > >= == !=</code>
逻辑	NOT AND OR	<code>! && </code>
界符	LPAREN RPAREN LBRACE RBRACE LBRACK RBRACK SEMI COMMA	<code>() { } [] ; ,</code>
结束符	EOF	输入结束标记

其中乘号在文法中写作 $\times$ ，而测试程序与 QTAC 中均使用 `*`，故实现以 `*` 为准

4.2.3 Token 数据结构

每个 Token 设计为一个四元组：

$$\text{Token} = (\text{type}, \text{value}, \text{line}, \text{col})$$

其中 `type` 为上表中的类型，`value` 为词素原文，`line` 与 `col` 记录该记号在源程序中的行号与列号。输出文件只取 `(type, value)` 两项；行列号则保留供后续阶段在报错时定位出错位置。

4.3 词法分析的 DFA 设计

4.3.1 总体思路

词法分析器的主体是一个 DFA。它遵循下面两条原则：

1. **最长匹配**：每次尽可能多地吞入字符，直到无法继续构成当前记号为止。
2. **跳过无意义字符**：空白符和行注释不产生记号，直接跳过。

4.3.2 顶层状态转移

设初始状态为 S_0 。在 S_0 处，先处理空白与注释，再依据当前首字符的类别分派到相应的子自动机：

$$S_0 \xrightarrow{\text{空白}} S_0 \quad S_0 \xrightarrow{\text{//} \cdots \text{行尾}} S_0$$

$$S_0 \xrightarrow{[a-zA-Z]} S_{id} \quad S_0 \xrightarrow{[0-9]} S_{num} \quad S_0 \xrightarrow{\text{运算符首字符}} S_{op} \quad S_0 \xrightarrow{\text{界符}} \text{接受}$$

若当前字符不属于上述任何一类，则报告词法错误。

4.3.3 各子自动机状态转移

1. 标识符/关键字：

吞入连续的字母，遇到非字母时接受，再用所得词素查关键字表，命中则为对应关键字，否则为标识符。

$$S_0 \xrightarrow{[a-zA-Z]} S_{id} \xrightarrow{[a-zA-Z]} S_{id} \xrightarrow{\text{非字母}} \text{接受 (查关键字表} \rightarrow \text{KEYWORD / ID)}$$

2. 整数常量：

吞入连续的数字，遇到非数字时接受为 NUM：

$$S_0 \xrightarrow{[0-9]} S_{num} \xrightarrow{[0-9]} S_{num} \xrightarrow{\text{非数字}} \text{接受 (NUM)}$$

3. 运算符：

$+ = < > !$ 均为“1 或 2 字符”运算符，靠前看一位区分； $\& |$ 必须成对， $/$ 还需与 $//$ 注释区分。

$$S_0 \xrightarrow{+} S_{+} : \quad \xrightarrow{=} \text{接受 PLUS_ASSIGN} ; \quad \text{否则接受 PLUS}$$

$$S_0 \xrightarrow{=} S_{=} : \quad \xrightarrow{=} \text{接受 EQ} ; \quad \text{否则接受 ASSIGN}$$

$$S_0 \xrightarrow{<} S_{<} : \quad \xrightarrow{<} \text{接受 LE} ; \quad \text{否则接受 LT}$$

$$S_0 \xrightarrow{>} S_{>} : \quad \xrightarrow{>} \text{接受 GE} ; \quad \text{否则接受 GT}$$

$$S_0 \xrightarrow{!} S_{!} : \quad \xrightarrow{!} \text{接受 NE} ; \quad \text{否则接受 NOT}$$

$$S_0 \xrightarrow{\&} S_{\&} : \quad \xrightarrow{\&} \text{接受 AND} ; \quad \text{否则报错}$$

$$S_0 \xrightarrow{|} S_{|} : \quad \xrightarrow{|} \text{接受 OR} ; \quad \text{否则报错}$$

$$S_0 \xrightarrow{/} S_{/} : \quad \xrightarrow{/} \text{转入行注释 (跳过)} ; \quad \text{否则接受 DIV}$$

$$S_0 \xrightarrow{-} \text{接受 MINUS} ; \quad S_0 \xrightarrow{*} \text{接受 MUL}$$

4. 界符:

$()\{\}\llbracket\rrbracket;$ 都是单字符记号, 读入即接受, 无中间状态:

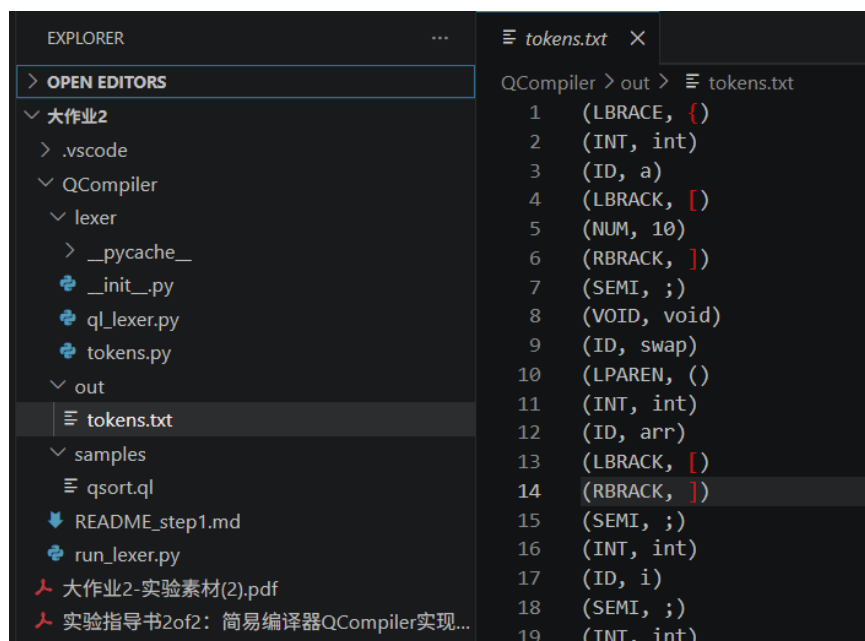
$$S_0 \xrightarrow{c} \text{接受} \quad (c \in , () \{ \} \llbracket \rrbracket ;)$$

4.4 编程实现与测试

我选用 Python 作为开发语言, 因为其处理字符串非常方便。整体程序分为三个模块: `tokens.py` 用于存放 Token 定义与规则表, `ql_lexer.py` 实现词法分析 DFA 的逻辑, `run_lexer.py` 是主驱动, 其读取 QL 源文件, 输出 Token 序列。整个数据流程是这样的:

QL源文件 $\rightarrow$ `Lexer.tokenize()` $\rightarrow$ Token列表 $\rightarrow$ 写出`tokens.txt`

测试输出如下:



```
QCompiler > out > tokens.txt
1  (LBRACE, {)
2  (INT, int)
3  (ID, a)
4  (LBRACK, [])
5  (NUM, 10)
6  (RBRACK, ])
7  (SEMI, ;)
8  (VOID, void)
9  (ID, swap)
10 (LPAREN, ())
11 (INT, int)
12 (ID, arr)
13 (LBRACK, [])
14 (RBRACK, ])
15 (SEMI, ;)
16 (INT, int)
17 (ID, i)
18 (SEMI, ;)
19 (INT, int)
```

图 1: Token Result

可以看到, 我们的词法分析器已经可以正确输出 Token 序列, 为之后的任务打下了良好的基础。

5 步骤二 (前半): SLR (1) 分析框架

5.1 任务概述

语法分析是编译流程的第二个阶段, 用词法分析的输出作为输入。它的任务是: 依据 QL 的文法, 把词法分析得到的 Token 流识别为符合语法结构的句子, 并构建出抽象语法树 QAST, 作为后续阶段的输入。

我们在这次实验中选择 B 路线, 采用 SLR(1) 自底向上分析: 先对 QL 文法做改造, 构造其 LR(0) 项目集规范族与 SLR(1) 分析表, 再以表驱动的移进-归约方式分析 Token 流, 并在归约时建立 QAST。

5.2 QL 文法的 SLR(1) 改造

素材给出的 QL 文法直接用于 SLR(1) 分析需要先做适配改造。本节说明改造的原则与各项调整。

5.2.1 改造原则

B 路线采用 SLR(1) 自底向上分析，其对文法的要求与 LL(1) 自顶向下分析不同。其**不必消除左递归、必须消除二义性**。所以我们依据实验要求，可对文法做必要修剪、或定义额外的冲突消解规则以满足 SLR(1)。

改造中，我对文法符号统一了命名。对于终结符，其必须采用词法器输出的 token 类型名，因为表驱动分析器是按每个 token 的”类型”去匹配文法符号的。对于非终结符，保留原 QL 文法的单字母名，便于与原文法对照。

5.2.2 表达式与条件的优先级分层

原文法的表达式 $E \rightarrow i \mid V \mid d(\check{R}) \mid EoE \mid (E)$ 把所有双目运算符笼统写作 EoE ，没有体现优先级与结合性，是二义的。改造的核心就是把它按优先级拆成若干层次：低优先级的运算符在上层，高优先级的在下层，每一层只处理一类运算符，从而用文法结构来唯一地确定运算顺序。

我们把表达式分为赋值、加减、乘除、因子的四层：

$$\begin{aligned} E &\rightarrow E_1 \text{ ASSIGN } E \mid E_1 \text{ PLUS_ASSIGN } E \mid E_1 \\ E_1 &\rightarrow E_1 \text{ PLUS } E_2 \mid E_1 \text{ MINUS } E_2 \mid E_2 \\ E_2 &\rightarrow E_2 \text{ MUL } E_3 \mid E_2 \text{ DIV } E_3 \mid E_3 \\ E_3 &\rightarrow \text{NUM} \mid V \mid \text{ID LPAREN } \check{R} \text{ RPAREN} \mid \text{LPAREN } E \text{ RPAREN} \end{aligned}$$

关于条件语句则是同理，原文法 $C \rightarrow E \mid ErE \mid !C \mid C\&\&C \mid C||C \mid (C)$ 也是二义的，我们还是用分层的思路去处理它：

$$\begin{aligned} C &\rightarrow C \text{ OR } C_1 \mid C_1 \\ C_1 &\rightarrow C_1 \text{ AND } C_2 \mid C_2 \\ C_2 &\rightarrow \text{NOT } C_2 \mid C_3 \\ C_3 &\rightarrow E \mid E \text{ LT } E \mid E \text{ LE } E \mid E \text{ GT } E \mid E \text{ GE } E \mid E \text{ EQ } E \mid E \text{ NE } E \end{aligned}$$

修改之后，E 与 C 由二义文法变为无二义文法。

5.2.3 其他修剪

除分层外，还做了以下几处调整以满足 SLR(1) 条件：

1. **关于分号**：原文法 $\check{D} \rightarrow \varepsilon \mid \check{D}\check{D};$ ， $\check{S} \rightarrow S \mid \check{S};S$ 把分号作为列表的分隔符，但这与 qsort 不符——其中变量声明、语句末尾自带;，而函数声明以函数体 } 结尾、后面并没有;。因此把终结符归入各声明/语句自身：变量声明写作 $D \rightarrow TV \text{ SEMI}$ 、表达式语句写作 $S \rightarrow E \text{ SEMI}$ 等。
2. **删除 (C)**：原文法中条件可加括号，但因 $C \rightarrow E$ 使任何表达式都是条件，(x) 既能按表达式括号解析、又能按条件括号解析，导致 SLR(1) 无法区分。所以我删去了 $C \rightarrow (C)$ 。

3. **arr[] 并入 V**: 原文法实参表的实参都是表达式 E, 无法表示 qsort 里以 arr[] 形式传递的数组。为此在左值中统一允许空方括号, 使 arr[] 自然成为一个表达式。
4. **保留悬空 else**: 原文法 $S \rightarrow if(C)S \mid if(C)S \text{ else } S$ 存在悬空 else 二义。改造中不对其改写, 而是保留两条产生式, 在构造分析表时对由此产生的移进-归约冲突采用”移进优先”消解。

5.2.4 完整文法

在上述修剪完成之后, 得到完整的文法如下:

```

1  P -> B
2  B -> LBRACE D̃ S̃ RBRACE
3  T -> INT | VOID
4  # 声明表
5  D̃ -> D̃ D | e
6  D -> T V SEMI | H B
7  H -> T ID LPAREN Ã RPAREN
8  # 左值
9  V -> ID | V LBRACK E RBRACK | V LBRACK RBRACK
10 # 形参表
11 Ã -> Ã A SEMI | e
12 A -> T ID | T ID LBRACK RBRACK
13 # 语句表
14 S̃ -> S̃ S | e
15 S -> SEMI | E SEMI | RETURN E SEMI
16     | IF LPAREN C RPAREN S
17     | IF LPAREN C RPAREN S ELSE S
18     | WHILE LPAREN C RPAREN S
19     | B
20 # 表达式
21 E -> E1 ASSIGN E | E1 PLUS_ASSIGN E | E1
22 E1 -> E1 PLUS E2 | E1 MINUS E2 | E2
23 E2 -> E2 MUL E3 | E2 DIV E3 | E3
24 E3 -> NUM | V | ID LPAREN R̃ RPAREN | LPAREN E RPAREN
25 # 实参表
26 R̃ -> R̃ E COMMA | e
27 # 条件
28 C -> C OR C1 | C1
29 C1 -> C1 AND C2 | C2
30 C2 -> NOT C2 | C3
31 C3 -> E | E LT E | E LE E | E GT E | E GE E | E EQ E | E NE E

```

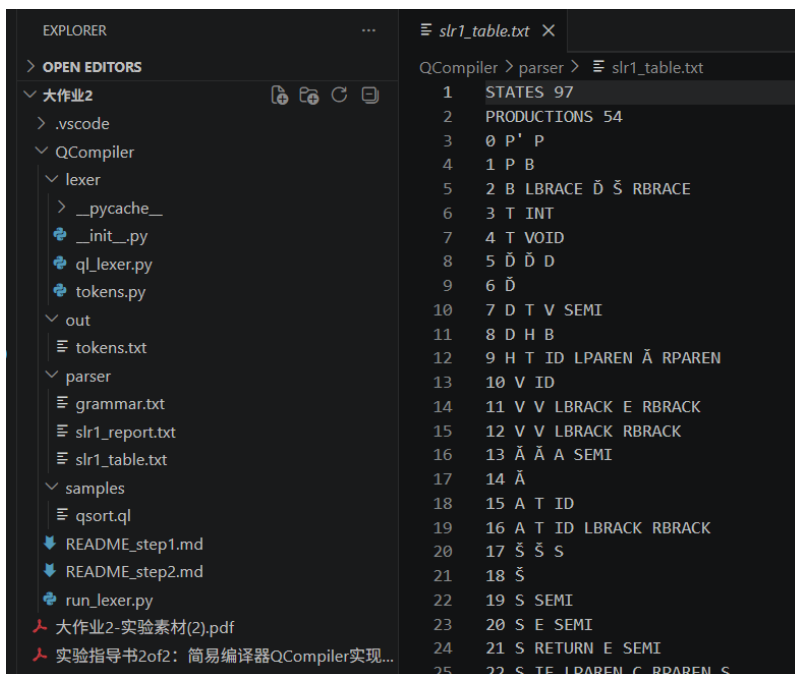
5.3 SLR(1) 分析表的构造与验证

5.3.1 构造流程

下面, 我们需要将改造后的 QL 文法构造成 SLR(1) 分析表, 大体分为添加增广文法、构造 LR(0) 项目集规范族、计算 FIRST 与 FOLLOW 集、填写 ACTION 与 GOTO 表四步。最后, 若同一表项被填入两个不同动作, 即出现冲突, 说明文法不是 SLR(1) 文法, 需调整文法或定义来消解规则。

5.3.2 生成 SLR(1) 分析表

SLR(1) 表的构造流程由一个通用的 SLR(1) 分析表生成器自动完成。该生成器复用自编译器设计实验课程的实验四 (slr1.cpp)，本实验将改造后的 QL 文法作为它的输入。运行命令为：slr1.exe parser/grammar.txt parser/slr1_table.txt。它读入文法文件，完成我们上面的四个步骤，最终将分析表以纯文本格式导出到 slr1_table.txt，供后续分析器读取。我们截取部分结果展示：



EXPLORER	slr1_table.txt
> OPEN EDITORS	QCompiler > parser > slr1_table.txt
大作业2	1 STATES 97
> .vscode	2 PRODUCTIONS 54
QCompiler	3 0 P' P
lexer	4 1 P B
> __pycache__	5 2 B LBRACE D S RBRACE
__init__.py	6 3 T INT
ql_lexer.py	7 4 T VOID
tokens.py	8 5 D D D
out	9 6 D
tokens.txt	10 7 D T V SEMI
parser	11 8 D H B
grammar.txt	12 9 H T ID LPAREN A RPAREN
slr1_report.txt	13 10 V ID
slr1_table.txt	14 11 V V LBRACK E RBRACK
samples	15 12 V V LBRACK RBRACK
qsort.ql	16 13 A A SEMI
README_step1.md	17 14 A
README_step2.md	18 15 A T ID
run_lexer.py	19 16 A T ID LBRACK RBRACK
大作业2-实验素材(2).pdf	20 17 S S S
实验指导书2of2: 简易编译器QCompiler实现...	21 18 S
	22 19 S SEMI
	23 20 S E SEMI
	24 21 S RETURN E SEMI
	25 22 S IF LPAREN C RPAREN S

图 2: SLR(1) Table

可以看到，成功生成了 SLR(1) 分析表，一共有 54 条产生式与 97 个 LR(0) 状态。

5.3.3 冲突的发现与消解

第一次运行之后，生成器报告了两处移进-归约冲突：

1.LBRACK 冲突： 这处冲突在设计文法时没有预料到。最初为了支持 qsort 里 arr[] 形式的数组实参，给实参单独加了一条产生式 $\rightarrow ID [] COMMA$ 。问题在于：它和普通的下标访问 a[i] 都以 ID [开头——当分析器读到 arr 准备读 [时，无法判断这个 arr 是要构成 arr[]，还是要构成 arr[i]。两种可能在同一状态下都成立，于是产生了移进-归约冲突。

修正办法是不再给实参特判，而在左值里统一允许空方括号 $V \rightarrow V []$ 。这样 arr[] 直接就是一个 V，分析器读到 arr[后只需看一个符号即可决定：是] 就当成 arr[]，是表达式开头就当下标访问，从而化解冲突。

2. 悬空 else 冲突： 这处冲突是预期之内、有意保留的。对读到 else 时”继续移进”还是”先归约 if(C)S”的二义，采用移进优先消解，即让 else 与最近的、尚未匹配的 if 结合。

至此，改造后的 QL 文法除有意保留的悬空 else 外，不存在其他冲突，是 SLR(1) 文法。

5.4 表驱动分析器的实现

有了 SLR(1) 表, 本节实现读取该表并对 Token 流进行移进-归约分析的驱动程序。使用 Python 实现。分析器的两个输入分别是 Token 流和产生的 SLR(1) 分析表两个文本文件。整个过程中, 分析器维护两个栈——状态栈与属性栈, 按当前状态 s 与当前输入终结符 a 查 $ACTION[s][a]$, 执行四类动作:

移进: 把目标状态 N 压入状态栈, 为该终结符构造叶节点压入属性栈;

规约: 取第 N 条产生式 $A \rightarrow \beta$, 从两个栈各弹出 $|\beta|$ 个元素, 构造 A 对应的 QAST 节点并压入属性栈, 再以 $GOTO[当前状态][A]$ 压入新状态;

接受: 分析成功, 属性栈顶即为 QAST 根;

空白表项: 报告语法错误。

当 $ACTION[s][a]$ 为空时, 说明输入不符合文法, 分析器报告语法错误, 并附带出错 token 的行列号、类型与词素。行列号由词法阶段为每个 token 记录, 便于定位源程序中的错误位置。

在这个阶段, 一共实现了四个.py 文件: 读入 SLR(1) 表 (slr_table.py)、规定一个 QAST 节点 (qast.py)、语义动作 (semantic.py)、移进-归约循环 (ql_parser.py)。对外接口为 `Parser(table).parse(tokens)`, 返回 QAST 根节点。

5.5 QAST 的设计与构建

本节再补充说明 QAST 节点的表示方式, 以及在归约时由语义动作建树的过程。

首先, 对于 QAST 节点, 素材为 QAST 定义了约 25 种具体结点, 我采用“通用节点 + 标签”的方式来实现在节点分类: 用一个统一的 ASTNode 类, 以 kind 字段标识节点种类, 而不为每种结点单独定义一个类。这样既覆盖了素材定义的全部节点分类, 又避免了为 25 种结点各写一个类带来的冗余, 遍历与打印都很方便。

其次, 关于由语义动作建树的过程, 每条产生式对应一个语义动作, 在该产生式归约时执行。语义动作按“左部 + 右部”来匹配产生式, 由弹出的子树构造左部对应的 QAST 节点。几类典型动作如下:

1. **二目运算 $\rightarrow$ 单个节点:** $E_1 \rightarrow E_1 \text{ PLUS } E_2$ 建 AOP(+), $C_3 \rightarrow E \text{ LT } E$ 建 ROP(<), 子节点为左右两棵子树; 运算符名取自该运算符 token 的词素。
2. **控制结构 $\rightarrow$ 专用节点:** $S \rightarrow \text{IF LPAREN } C \text{ RPAREN } S [\text{ELSE } S]$ 建 If (含/不含 else 分支)、 $S \rightarrow \text{WHILE} \dots$ 建 While、 $S \rightarrow \text{RETURN } E \text{ SEMI}$ 建 Return。
3. **函数调用与数组访问:** $E_3 \rightarrow \text{ID LPAREN } \check{R} \text{ RPAREN}$ 建 Call(name), $V \rightarrow V \text{ LBRACK } E \text{ RBRACK}$ 建 ArrayAccess (带下标)、 $V \rightarrow V \text{ LBRACK } \text{RBRACK}$ 建空括号的 ArrayAccess([]) (数组传名)。
4. **声明与函数定义:** $D \rightarrow T V \text{ SEMI}$ 建 VarDecl、 $H \rightarrow T \text{ ID LPAREN } \check{A} \text{ RPAREN}$ (连同 $D \rightarrow H B$) 建 FuncDecl, 并记录返回类型与形参。

经上述语义动作, 分析器在归约的同时逐层建起 QAST, 接受时属性栈顶即为完整的语法树根。

5.6 QAST 输出测试

以快速排序程序 qsort.ql 为测试用例, 运行驱动程序, 截取的部分输出如下:

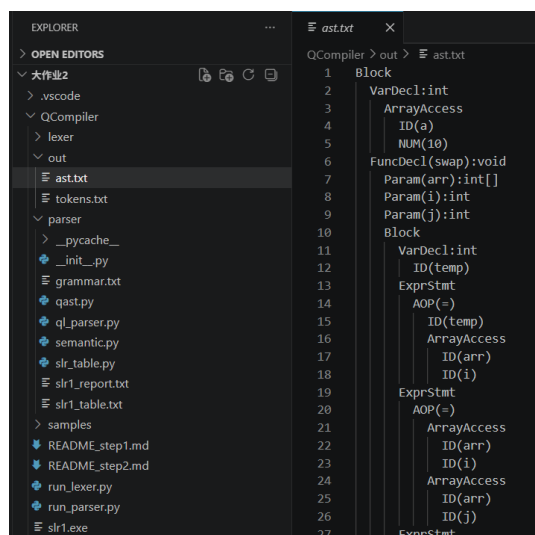


图 3: QAST Result

检查之后也发现 QAST 生成无误。综上，改造后的 QL 文法在 SLR(1) 分析下能正确分析程序，表驱动分析器与语义动作成功构建出与源程序对应的 QAST。所得 QAST 将作为下一阶段中间代码生成的输入。

6 步骤二 (后半): 中间代码生成 (符号表 + QTAC)

6.1 任务概述

中间代码生成的任务是遍历语法分析得到的 QAST，生成 QTAC 三地址代码，同时建立符号表。本阶段完成后，源程序就可以转化为三地址指令序列，作为后续 ARM 目标代码生成的输入。

本阶段采用语法制导翻译：对 QAST 自顶向下遍历，对表达式结点生成求值代码并返回其结果位置，对语句结点直接发射相应指令，在遍历过程中同时生成符号表。素材给出了 qsort 程序的参考 QTAC，本阶段以它为对照标准，验证所生成中间代码的正确性。

6.2 符号表设计

符号表用于在中间代码生成过程中登记程序中的各类名字，本实验采用作用域栈：栈底为全局作用域，每进入一个函数体便压入一层函数作用域，函数处理完毕即弹出。每个符号条目录录以下信息：

表 2: 符号表字段

字段	含义
名字	变量名 / 函数名
种类	全局变量 / 函数 / 形参 / 局部变量
类型	int / void (数组在类型上标注 [])
作用域	所属作用域
数组大小	数组元素个数

以 `qsort` 为例，符号表登记了全局数组 `a`、三个函数 `swap/partition/qsort`（以及各自的返回类型与形参类型）、以及各函数内部的形参与局部变量。

6.3 QTAC 指令与表示

按素材的 QTAC 文法，中间代码由以下几类指令构成：

表 3: 四元式指令形式

类别	指令形式	含义
全局声明	<code>i<width> d;</code>	全局变量/数组， <code>width</code> 为字节宽度
函数框架	<code>define d(a1,...){ LABEL d; ... }</code>	函数定义，入口标号为函数名
控制流	<code>LABEL l; GOTO l; IF q rop a THEN l ELSE l;</code>	标号、无条件转移、条件转移
复制	<code>q = a;</code>	赋值
二目运算	<code>q = a op b;</code>	算术运算，也用于地址计算 <code>q = base + idx</code>
访存	<code>q = M[a]; M[q] = a;</code>	取数、存数
函数调用	<code>PAR a; q = CALL d, k;</code>	传参、调用
返回	<code>RETURN a;</code>	函数返回

实现中，我把指令格式集中在 `qtac.py` 文件，为上述每类指令提供一个发射方法，方法内部负责拼出符合 QTAC 文法的文本并追加到指令序列。所有指令收集完毕后输出的文本即为 QTAC 代码。

6.4 遍历生成 QTAC 代码和符号表

本节是中间代码生成的核心：对 QAST 自顶向下遍历，按结点种类执行相应的语义动作生成 QTAC。

6.4.1 遍历规则

遍历遵循三条统一规则：

1. 对表达式结点生成求值代码后，返回一个保存其结果的”位置”——临时变量名、变量名或常量字面量，供上层指令引用。
2. 对语句结点（赋值、if、while、return、调用等）直接发射对应的 QTAC 指令。
3. 临时变量与控制流标号在整个程序范围内连续分配，与素材的编号风格一致。

6.4.2 表达式与赋值语句生成

关于二目运算 (`a op b`)，我的策略是统一生成到一个新临时变量。先递归求值左右操作数得到位置 `a'`、`b'`，再发射 `t = a' op b'`，返回 `t`。

关于赋值，我采用的是”目标驱动”策略，具体说就是根据右值的形态决定结果写到哪里：

表 4: 赋值对应的 QTAC 生成

赋值形态	生成的 QTAC
<code>x = 函数调用</code>	<code>...PAR...; x = CALL f, k</code> (结果直接写入 <code>x</code>)
<code>x = arr[i]</code>	<code>tS = i*8; t = arr + tS; x = M[t]</code> (取数直接写入 <code>x</code>)
<code>x = a op b</code>	<code>t = a op b; x = t</code> (运算先入临时, 再复制)
<code>arr[i] = e</code>	<code>tS = i*8; t = arr + tS; < 求值 e>; M[t] = p</code> (先算地址, 再存数)

6.4.3 控制流语句生成

关于控制流语句, 条件结点并不是计算出一个布尔值再判断, 而是直接接收”条件为真该去哪”和”为假该去哪”两个出口标号, 生成跳转把控制送到对应位置。

具体而言, 关系比较只需一条 `IF a rop b THEN 真出口 ELSE 假出口`; `if` 语句把”真出口”接到语句体入口、”假出口”接到语句体之后; `while` 语句则需要在此基础上多做一件事——在循环体末尾加一条回到条件判断处的跳转。带 `else` 的 `if` 则为两个分支各设标号, 并在真分支末尾跳过假分支。

值得强调的是赋值作条件的处理。由于”表达式求值会先发射其副作用代码、再返回结果位置”这一规则, 当条件中含有赋值时, 赋值的副作用 (`j` 自增) 会在比较指令之前完成, 比较再用赋值的结果参与。我们无需为这种情形写特殊逻辑, 它是遍历约定的自然结果。

6.4.4 函数、调用与声明

这三类的共同思路是, 把”需要落到指令的”(运算、传参、调用、返回)与”只需记录的”(局部变量的存在、类型信息)分开处理——前者发射 QTAC, 后者交给符号表。

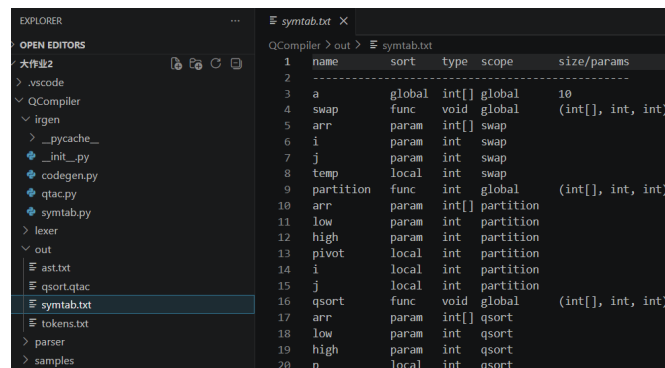
关于函数定义, 我们生成函数框架后遍历函数体。这里的关键是局部变量只登记符号表、不发任何指令。因为在 QTAC 层面局部变量就是具名变量, 无需分配指令;

关于函数调用, 逻辑要点在求值与传参的次序, 需要先把各实参逐一求值再统一传参, 最后发射调用。

关于声明, 全局变量声明由符号表的元素个数与元素字节宽度算出总宽度, 局部变量声明则不产生代码。

6.4.5 符号表与 QTAC 代码输出测试

以快速排序程序 `qsort.ql` 为测试用例, 运行驱动程序, 测试输出结果如下:



name	sort	type	scope	size/params
a	global	int[]	global	10
swap	func	void	global	(int[], int, int)
arr	param	int[]	swap	
i	param	int	swap	
j	param	int	swap	
temp	local	int	swap	
partition	func	int	global	(int[], int, int)
arr	param	int[]	partition	
low	param	int	partition	
high	param	int	partition	
pivot	local	int	partition	
i	local	int	partition	
j	local	int	partition	
qsort	func	void	global	(int[], int, int)
arr	param	int[]	qsort	
low	param	int	qsort	
high	param	int	qsort	
p	local	int	qsort	

图 4: 符号表输出

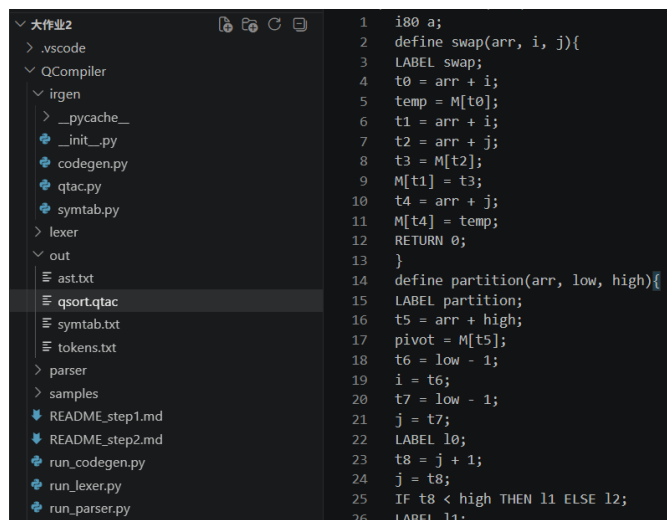


图 5: QTAC 输出

将生成结果与素材给出的参考 QTAC 逐条对照，整体结构完全一致，但是有三处不同：

1. **数组下标按元素宽度缩放：**参考 QTAC 的数组寻址写作 `arr + i`，未乘以元素宽度。这在字节寻址的 ARM 上会让 8 字节元素互相重叠访问。在测试的时候，我发现直接照此生成的程序排序结果错误。定位后将下标按元素宽度缩放为 `arr + i*8`，重新测试排序正确。
2. 修正了参考 QTAC 的一处笔误：参考中 `partition` 末尾对 `swap(arr[], i+1, high)` 的翻译为 `t13 = arr + j;`，此处应是笔误，正确生成应是 `t13 = i + 1; PAR arr; PAR t13; PAR high;`。
3. 对循环体内无 `else` 的 `if`，参考在真分支末尾与假分支处各放一条 `GOTO l0`；本实现让两条路径汇合到 `if` 之后的同一条 `GOTO l0`，二者语义等价。

7 步骤三：QTAC→ARM64 汇编生成

7.1 任务概述

目标代码生成是编译流程的最后一个核心阶段，承接中间代码生成的输出，把与架构无关的 QTAC 三地址码翻译为可在华为鲲鹏服务器上运行的汇编代码。本阶段以 `qsort.qtac` 为输入，生成 `qsort.s` 汇编文件。

本阶段主要复用大作业 1 已实现的架构，按 Q2ARM 指令模板把 QTAC 映射为 ARM 指令。DFA 的构建原理、Q2ARM 模板的映射方式已在大作业 1 中详述，此处不再展开，仅说明为支持 `qsort` 而做的扩展。

为此，本阶段在大作业 1 的基础上做了下面的四方面扩展。

7.2 寄存器分配

大作业 1 中，`fact` 的几个变量固定占用某些寄存器，但这套办法处理不了 `qsort`，因为它的变量数量远超过可用寄存器的数量，而且很多值要在函数调用前后保持不变。为此本阶段改用栈槽分配，思路很简单：每个 QTAC 变量分一个栈上的槽位，所有值都存在栈里。常量、标号、全局数组名不是变量，不占槽。

这样做的好处是我们无需关心哪些寄存器会被调用者破坏。代价是 `load/store` 较多、代码不够简洁。

在此约定下，每条指令的操作数都先从其槽位加载到寄存器 x9/x10，运算后结果再存回目标槽。操作数按三种情形加载：

表 5: 操作数加载方式	
操作数类型	加载方式
常量 k	mov reg, #k
全局数组名 a	adr reg, a (取其基址)
变量 / 临时	ldr reg, [sp, # 槽偏移]

每个函数所需的栈空间由它用到的变量个数决定，并对齐到 16 字节以满足 ARM64 的要求。

7.3 函数与调用约定

支持多函数与调用，需要为每个函数建立规范的栈帧，并遵循 AAPCS64 调用约定。

1. 函数帧：每个函数进入时执行序言、退出时执行尾声：序言先保存帧指针与返回地址（stp x29, x30），再将栈指针下移、为局部槽位开辟空间（sub sp）；尾声则反向回收空间并恢复（add sp、ldp、ret）。

2. 函数调用：先把各实参依次加载到参数寄存器 x0–x7，再 bl f 跳转调用，返回后结果在 x0，存回结果变量的槽位。由于本阶段所有值都在栈上，调用过程即使破坏了寄存器也不影响，调用从槽位重新加载即可。

3. 返回：return E 翻译为把返回值加载到 x0，再跳转到本函数尾声处的统一出口标号，由尾声完成栈回收与返回。void 函数则以默认的返回 0 收尾。

7.4 内存访问与全局数组

qsort 大量使用数组，因此本阶段新增了内存访问与全局数组的处理。

1. 内存访问：QTAC 的取数与存数直接对应 ARM 的 ldr/str。数组元素的访问被拆成“先算地址、再访存”两步，这与 QTAC 在中间代码阶段就已拆好的形式对应，目标代码只需逐条翻译。

2. 全局数组：全局数组在数据段中以 .skip 预留空间声明，其字节数即中间代码里算好的宽度。当全局数组名作为值出现时（如把数组传给函数），取的是它的基址——用 adr 指令把该数组标号的地址加载到寄存器。

7.5 输出测试

对 qsort.qtac 运行目标代码生成器，部分结果截图如下：

```

1 .section .data
2 .align 3
3 at: .skip 80
4 .section .text
5 .global _start
6 swap:
7 stp x29, x30, [sp, #-16]!
8 mov x29, sp
9 sub sp, sp, #80
10 str x0, [sp, #0]
11 str x1, [sp, #8]
12 str x2, [sp, #16]
13 ldr x9, [sp, #0]
14 ldr x10, [sp, #8]
15 add x9, x9, x10
16 str x9, [sp, #24]
17 ldr x9, [sp, #24]
18 ldr x10, [sp, #8]
19 str x10, [sp, #32]
20 ldr x9, [sp, #0]
21 ldr x10, [sp, #8]

```

图 6: Result

至此，目标代码生成阶段完成，QTAC 被正确翻译为可在鲲鹏平台运行的 ARM64 汇编。

8 步骤四：ARM 汇编代码优化

8.1 任务概述

代码优化的目标是在保持程序语义不变的前提下，消除冗余、提升生成代码的质量。之所以有优化空间，是因为前面各阶段为求清晰、正确，生成的代码留有明显冗余。例如中间代码里同一地址 `arr+i` 被重复计算、`t = expr; x = t` 这类多余复制的出现等情况。这些都可以在不改变语义的情况下消去。

我们从指导书给的可选优化点中选择几项，并分为两层实施：

1.QTAC 中端优化：公共子表达式消除、复制传播、死代码删除。

2.ARM 后端优化：针对具体的 ARM 指令排布，进行冗余跳转消除。

8.2 基本块划分

复制传播与公共子表达式消除若直接在整个函数上做，遇到分支、循环等语句就可能出错。例如在两条路径汇合处，某个变量从不同来路带来的值并不相同，跨过去替换就会改变语义。为避免这一问题，本阶段把这两项优化限制在基本块内进行。

基本块就是单入口、单出口的直线指令段：控制流只能从块首进入、从块尾离开，中途不会跳入也不会跳出。块内是顺序执行的直线代码，分析起来简单，在块内做局部优化很安全。

划分时先确定各基本块的块首：函数的第一条指令是块首，所有 LABEL 是块首，GOTO/IF/RETURN 之后的那条指令也是块首。每个块首到下一个块首之间的指令构成一个基本块。划分完成后，对每个基本块独立做复制传播与公共子表达式消除；而死代码删除因依赖整个函数的“使用”信息，则在合并回完整函数体后统一进行。

8.3 中端优化：公共子表达式消除、复制传播与死代码删除

这三项优化在 QTAC 上协同工作：公共子表达式消除发现重复计算并把它改写为复制，复制传播把这些复制顺出去，死代码删除再把变成无用的指令清除。下面以 `qsort` 为例说明：

公共子表达式删除：在基本块内维护一张“可用表达式表”，记录每个算式 `a op b` 的结果存在哪个临时里。当同一算式再次出现且其操作数中途未被改变时，便把后一次计算改写为对前一次结果的复用。以 `swap` 为例，计算 `arr[i]` 的地址需 `i*8` 与 `arr+i*8` 两步，其中下标缩放 `i*8` 被算了两次：

```
1  t0 = i * 8;          t0 = i * 8;
2  t1 = arr + t0;        t1 = arr + t0;
3  temp = M[t1];         temp = M[t1];
4
5  t2 = i * 8;          →  t2 = t0;          ← i*8 已在 t0, 复用, 不再重算
6
7  t3 = arr + t2;        t3 = arr + t2;
8  ...                  ...
9  M[t3] = t6;          M[t3] = t6;
```

需要注意，只对纯算术表达式做此优化，`M[]` 取数不参与，因为其内存可能已被改写。

复制传播：它把形如 $x = y$ 的复制传播下去：在 x 被重新赋值之前，把后续对 x 的引用都替换为 y 。接着上面产生的 $t2 = t0$ ，复制传播把 $t3 = arr + t2$ 改写为 $t3 = arr + t0$ ：

```
1  t2 = t0;          t2 = t0;
2
3  t3 = arr + t2; →   t3 = arr + t0;    ← t2 换成 t0
```

复制传播在基本块内进行，因而总是安全的。

死代码删除：QTAC 的临时变量都是单赋值的，若某临时从头到尾未被任何指令读取，它就是死代码，可以删去。该过程迭代进行，直到不再有可删指令。例如，经过前两步， $t2 = t0$ 被删除：

```
1  t0 = i * 8;
2  t1 = arr + t0;
3
4  temp = M[t1];      ← t2 = t0 被删除
5
6  t3 = arr + t0;
7  M[t3] = t6;
```

此外，死代码删除还能处理结果未被使用的函数调用。

综合这三步，`swap` 中 `arr[i]`、`arr[j]` 的地址各只算一次，`partition` 中重复的 `low-1`、`i+1` 也被消去，多处多余复制被清除。它们都在保持语义不变的前提下减少了指令。

8.4 后端优化：冗余跳转消除

前述三项优化都在与机器无关的 QTAC 上完成，但是生成 ARM 后，汇编层面还会有一类冗余出现。若一条无条件跳转 `b IX`，其目标 `IX` 恰好就紧跟在它后面，既然不跳转也会顺序落到 `IX`，这条跳转就是多余的，可以直接删去。这类问题只在具体的 ARM 指令排布上才看得出来，故放在后端优化处理。

以函数尾声为例。步骤三为简单，把 `return` 统一翻译为“跳转到本函数出口标号”，于是最后一条 `return` 生成的 `b swap_exit` 后面正好紧跟出口标号 `swap_exit:`：

```
1  mov x0, #0      mov x0, #0
2  b swap_exit →   swap_exit:      ← 删去 b swap_exit, 落空即可
3 swap_exit:      add sp, sp, #64
4  add sp, sp, #64 ...
```

对 `qsort` 一共消除了 3 条这样的跳转，实现了最终代码的精简。

8.5 结果验证

对 `qsort` 运行优化程序，优化前后的对照如下：

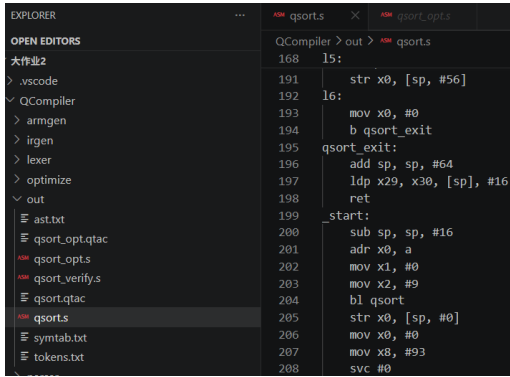


图 7: 优化前

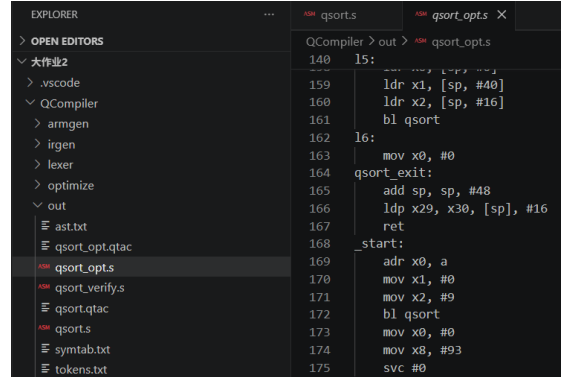


图 8: 优化后

可以看到，ARM 汇编行数从 208 减少为了 175，且确认了语义是一致的。综上，本阶段在不改变程序功能的前提下，有效精简了中间代码与目标代码，代码优化阶段完成。

9 步骤五：整体测试与鲲鹏环境验证

全链路的验证我们实际上已经测试到生成 ARM 代码这一步了，实际上我们现在只需测试生成的 ARM 代码能否在鲲鹏服务器上正确运行即可。

接下来我们把汇编文件上传到服务器，先初始化测试数组：

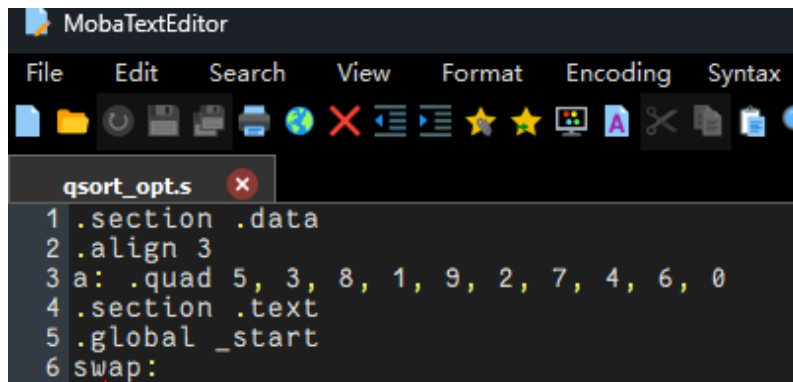


图 9: 测试数组初始化

接下来，我们汇编、链接、运行，结果如下：

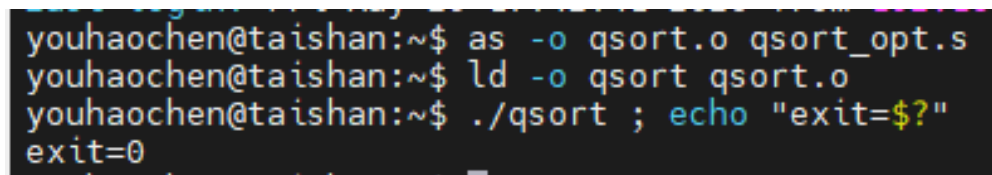


图 10: Result

exit=0 证明程序正常运行完成了，接着我们用 gdb 进行单步调试，看看流程对不对：

```

(gdb) break _start
Breakpoint 1 at 0x4002c4
(gdb) run
Starting program: /home/youhaochen/qsort

Breakpoint 1, 0x000000004002c4 in _start ()
(gdb) x/10dg &a
0x4102e0:      5      3
0x4102f0:      8      1
0x410300:      9      2
0x410310:      7      4
0x410320:      6      0

```

图 11: gdb Result

可以看到，我们的初始化数组被正确都出来了，现在让排序跑完，再看一次结果：

```

(gdb) ni
0x00000000400308 in _start ()
(gdb) ni
0x0000000040030c in _start ()
(gdb) ni
0x00000000400310 in _start ()
(gdb) ni
0x00000000400314 in _start ()
(gdb) ni
0x00000000400318 in _start ()
(gdb) x/10dg &a
0x410320:      0      1
0x410330:      2      3
0x410340:      4      5
0x410350:      6      7
0x410360:      8      9

```

图 12: Result after qSort

现在已经完全排序好了！证明我们生成的 ARM 代码可以在鲲鹏服务器上运行。实验到这里全部完成！

10 个人总结

10.1 AI 使用总结

本次实验中，我把 AI 作为学习与开发的协作工具。AI 主要承担两类工作：一是讲解 SLR(1) 文法改造、ARM64 调用约定等原理，帮助我理解各阶段的来龙去脉；二是在关键设计上为我列出可选方案与利弊，供我权衡——A/B/C 三条路线最终选 B、寄存器分配方法、步骤四做哪几项优化、分析器的实现方法，这些决策都由我在对比后自己确定。整个过程中在 AI 辅助之余，我认为也需坚持“先看懂再接受”的原则，确保每一处实现我都能讲清逻辑，同时也复用并扩展了自己前期的成果（实验四的 SLR 分析表生成器、实验五的表驱

动分析器、大作业一的代码生成器)。整个实验让我体会到, AI 能加速从理论到实现的落地, 但真正保证正确、做出取舍, 仍要靠自己的理解与动手验证。

10.2 收获与展望

通过本次实验, 我完整地走通了一条编译器的全流程: 从 QL 源程序的词法分析, 到 SLR(1) 语法分析与抽象语法树构建, 再到 QTAC 中间代码生成、ARM64 目标代码生成、代码优化, 直至在鲲鹏平台汇编、链接、运行并验证。这个过程把课堂上相对零散的知识点真正串了起来。原本停留在书本上的”前端—中端—后端”分工, 这一次都变成了可以运行、可以调试的代码, 理解也因此从”知道概念”深化为”讲得清、写得出”。在工程层面, 我也体会到模块化与”数据—逻辑分离”的价值: 每个阶段职责单一、以明确的中间产物为接口, 使得整条流水线既能逐段验证, 也便于复用和扩展。总而言之, 这是一次收获颇丰的实验。